

PolarNav AI — Judge Guide (SIH26059)

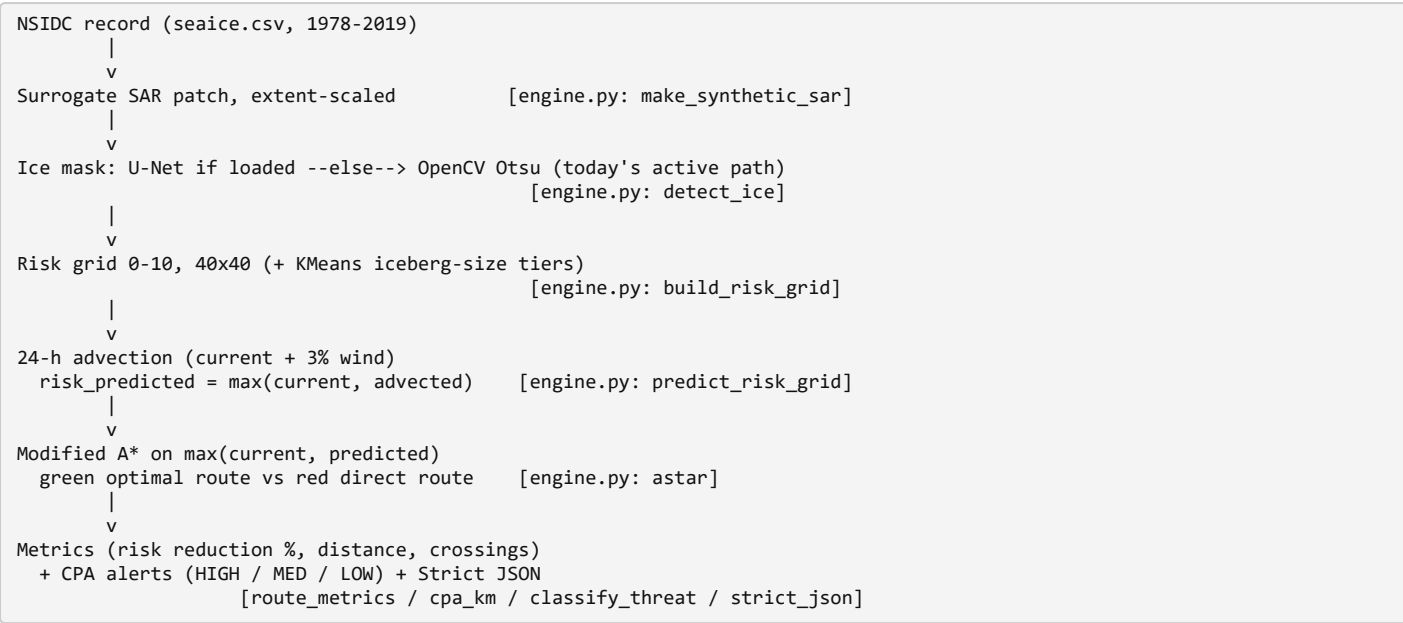
Offline Antarctic ice-navigation decision support — no black boxes.

Model status (read first)

Vision: SmallUNet exists and is architecture-verified, but is gated behind a Dice bar (≥ 0.60 and $\geq \text{Otsu} + 0.05$) it has not yet cleared — Otsu is the active path today. Drift: the Ridge model is trained on synthetic samples that reproduce the physics formula — this certifies the pipeline, not real-world drift skill. KMeans: unsupervised risk-tier clustering, not a prediction model.

The system in one breath

A real NSIDC extent record drives a synthetic SAR patch, segmented into an ice mask, built into a 0-10 risk grid, advected 24 h by wind/current, routed with a risk-penalized A*, and reported as metrics, alerts, and JSON.



Repo map

File	What it is
engine.py	All math: segmentation, risk, drift, A*, metrics, alerts, JSON. No UI.
app.py	Streamlit UI + session_state only; computes nothing itself.
train_unet.py	Notebook-faithful GPU/CPU trainer for SmallUNet. Not run by the app.
train_drift.py	Trains the Ridge drift model. Not run by the app.
models/	drift_model.joblib trained. unet_weights.pth absent, not yet trained.
data/	Satellite drops (icebergs.csv, wind_current.csv), coast.geojson, SAR sample.
seaice.csv	Real NSIDC record; drives the surrogate SAR's ice density.
docs/training_evidence/	Screenshots and logs from this build's QA passes.
.streamlit/config.toml	Disables rerun-on-save bug; serves the offline map's assets.

Run & click

setup_demo.bat	(once, with internet - installs pinned deps)
demo.bat	(every demo - offline, system Python, no venv)

Click	What appears	Proves
Simulate Satellite Data Drop	3 receipts + real NSIDC date/extent badge	F1, data-driven surrogate
Detect Ice	SAR + mask side by side, honest "Active model" caption	F2, segmentation
Compute Risk-Aware Route	5 metrics, green vs red route, blue overlay	F3-F6
(same screen)	Live Alerts panel, HIGH/MED/LOW	CPA alerting
Expand Strict JSON	NCPOR-format payload	F9

Read before asking about accuracy

Pixel accuracy is a trap here: a frame 90% open water scores 90% accuracy by predicting "all water," missing every iceberg. We report Dice and IoU instead:

$$\text{Dice} = 2|P \cap T| / (|P| + |T|) \quad \text{IoU} = |P \cap T| / |P \cup T|$$

The reference notebook's 8-class RGB baseline scores **IoU 0.441** (notebook, final cell). Our binary SAR model is untrained: zero labelled patches exist here, so no Dice/IoU exists yet. Shipping is gated on **Dice ≥ 0.60 AND $\geq \text{Otsu} + 0.05$** on held-out patches (`train_unet.py`), never a weaker bar. Live and reproducible today: **71.7% risk reduction, +38.2% distance cost** (`python engine.py self-test`).

Honesty legend

- "Active model: OpenCV Otsu (fallback)" — today's real state, always reachable, no training data required.
- "Active model: SmallUNet (trained weights)" — appears only once a model clears the Dice bar above.
- NSIDC-driven surrogate SAR — blob density scales with a real historical extent (badge shows the date), not an arbitrary image.
- Ridge drift model — trained on synthetic samples reproducing the physics formula; proves the pipeline, not real-world skill.
- Not claimed today: an LSTM/sequence drift model. That is Phase 2.

```
torch installed?
NO --> Otsu (always-on fallback)
YES --> models/unet_weights.pth present?
      NO --> Otsu
      YES --> mask coverage within [1%, 60%]?
            NO --> Otsu (sanity guard rejects a broken model)
            YES --> U-Net mask used; caption flips automatically
```

```
Labelled patches: data/train_patches/{images,masks}/
                  | train_unet.py (GPU if available, notebook-faithful recipe)
                  v
Held-out Dice(SmallUNet) vs Dice(Otsu), same split
                  |
                  v
Dice  $\geq 0.60$  AND  $\geq \text{Otsu} + 0.05$  ?
NO --> weights NOT saved, Otsu stays active
YES --> unet_weights.pth saved --> engine.py loads it --> caption flips
```

PPT crosswalk

Slide	Demo moment	File
Problem & data	NSIDC badge on data-drop	<code>seaice.csv</code>
Detection	Ice mask + honest caption	<code>engine.py: detect_ice</code>
Risk model	Risk grid + overlay	<code>build_risk_grid, predict_risk_grid</code>
Routing	Green vs red route, metrics	<code>astar, route_metrics</code>
Safety	Live Alerts panel	<code>cpa_km, classify_threat</code>
Integration	Strict JSON expander	<code>strict_json</code>

FAQ

- **Why offline?** Ships lack reliable connectivity; the demo proves it works with Wi-Fi off.
- **Why A* over ML routing?** Deterministic and explainable to a captain in seconds.
- **Why not LSTM drift yet?** Needs a real trajectory dataset we lack; physics + Ridge is the honest interim.
- **Quantization cost?** Not yet measured — no trained U-Net exists.
- **What breaks first in production?** Real Sentinel-1 imagery quality, not the algorithm.
- **Fuel model?** Distance delta of the routed path vs. the direct line.
- **CPA tiers?** HIGH under 5 km or a predicted risk > 5 crossing, MED under 10 km, else LOW.
- **How do updates reach the ship?** Out of scope for this decision-support demo.

Demo-day numbers (live, reproducible via `python engine.py`)

Risk reduction **71.7%** · Distance cost **+38.2%** · Grid **40x40 @ ~2.775 km/cell** · Drift model $R^2 \sim 1.00$ (**synthetic**) · NSIDC record **1978-2019, 13,177 days** · U-Net status **not yet trained (0 patches)**